

【uboot】uboot 2020.04 Pinctrl子系统分析和使用

原创

ZHONGCAI0901

于 2021-06-18 11:02:00 发布

阅读量3.4k

收藏 29

点赞数 6

分类专栏：[uboot](#)

文章标签：[uboot](#)

版权

 uboot 专栏收录该内容

5 篇文章

订阅专栏

相关文章

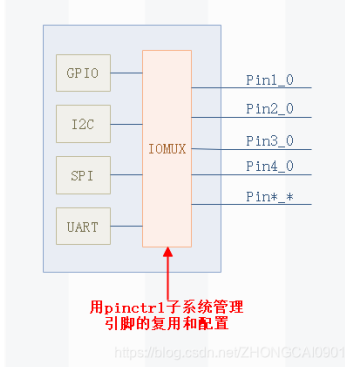
1. 《【uboot】imx6ull uboot 2020.04源码下载和编译环境配置》
2. 《【uboot】uboot 2020.04 DM驱动模式 – Demo体验》
3. 《【uboot】uboot 2020.04 DM驱动模式 – 架构分析》

1. 前言

在许多soc内部都包含有pin控制器，通过pin控制器的寄存器，我们可以配置一个或者一组引脚的功能和特性。uboot提供一个类似linux 的pinctrl子系统，目的是为了统一各soc厂商的pin脚管理。

2. Pinctrl子系统的功能简介

我们可以通过**pinctrl子系统**来设置引脚的复用、配置，可以将IO复用成 **GPIO** 、**I2C**等其它功能。图 下：



- 管理系统中所有的可以控制的pin，在系统初始化的时候，枚举所有可以控制的pin，并标识这些pin。
- 管理这些pin的复用（Multiplexing）。对于SOC而言，其引脚除了配置成普通的GPIO之外，若干个引脚还可以组成一个pin group，行程特定的功能。pin control subsystem要管理所有的pin group。
- 配置这些pin的特性。例 使能或关闭引脚上的pull-up、pull-down电阻，配置引脚的driver strength。

3. Pinctrl Uclass创建流程

下面是**pinctrl uclass**创建和绑定的过程 下：

```
1 [root.c]dm_extended_scan_fdt(gd->fdt_blob, false); // 在设备树中搜索设备并进行驱动匹配，然后bind。
2 [root.c]dm_scan_fdt(gd->fdt_blob, false); // 扫描设备树并绑定驱动程序
3 [root.c]dm_scan_fdt_node(gd->dm_root, gd->fdt_blob, 0, false); // 扫描设备树并绑定一个节点的驱动程序
4 [lists.c]lists_bind_fdt(gd->dm_root, offset_to_ofnode(0), NULL, false); // 绑定设备树节点，它会给绑定的设备树节点创建一个新udevice，并使用@parent作为其父设备。
5 [device.c]device_bind_with_driver_data(parent, entry, name, id->data, node, &dev); // 创建一个设备并且绑定到driver。
6 [device.c]device_bind_common(parent, drv, name, NULL, driver_data, node, 0, devp); // 创建一个设备并且绑定到driver。
7 [uclass.c]uclass_get(drv->id, &uc); // 根据ID获取一个uclass， 果它不存在就创建它。
8 [uclass.c]uclass_find(); // 根据id找到对应的uclass。
9 [uclass.c]uclass_add(); // 在未找到的情况下，就会在列表中创建一个新的uclass。
10 [device.c]calloc(1, size); // 1: 创建一个新的udevice并初始化； 2: 给dev->platdata, dev->uclass_platdata, dev->parent_platdata分配空间
11 [device.c]list_add_tail(&dev->sibling_node, &parent->child_head); // 将新dev放到父节点的列表中
12 [uclass.c]uclass_bind_device(dev); // 将device与uclass进行绑定，设备连接到uclass的设备列表中。
13 [U_BOOT_DRIVER]drv->bind(dev); // device绑定成功后，就会调用drv->bind。
14 [U_BOOT_DRIVER]parent->driver->child_post_bind(dev); // 在一个新的child被绑定后，就会调用parent driver的child_post_bind(dev);
15 [UCLASS_DRIVER]uc->uc_drv->post_bind(dev); // 在一个新设备绑定到这个uclass后被调用
```

经过以上的流程，它会将**pinctrl**的**uclass**、**uclass_driver**、**udevice**、**driver**进行绑定，绑定后的关系 下图所示：

登录后您可以享受以下权益：

 免费复制代码

 和博主大V互动

 下载海量资源

 发动态/写文章/加入社区

觉得还不错？[一键收藏](#)

 ZHONGCAI0901

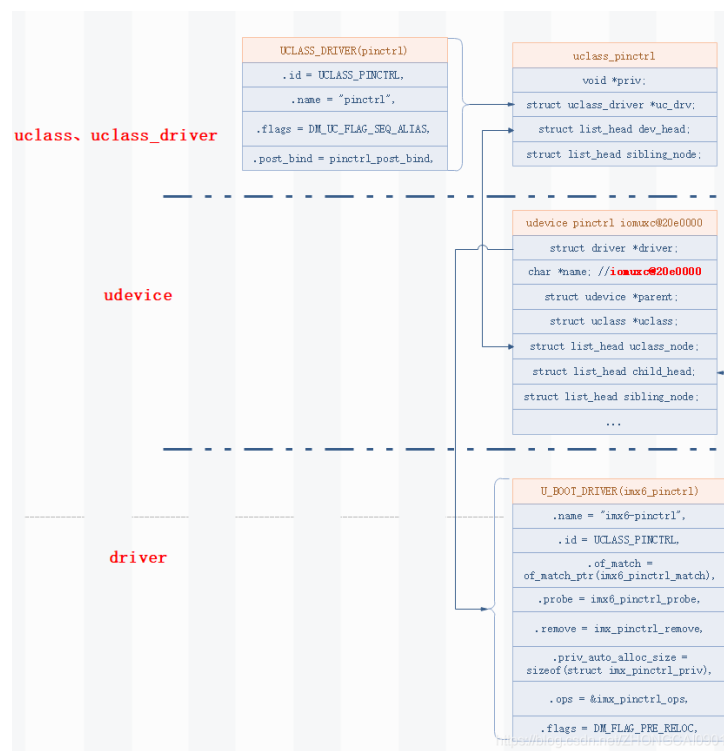
关注

6

29

1

分享

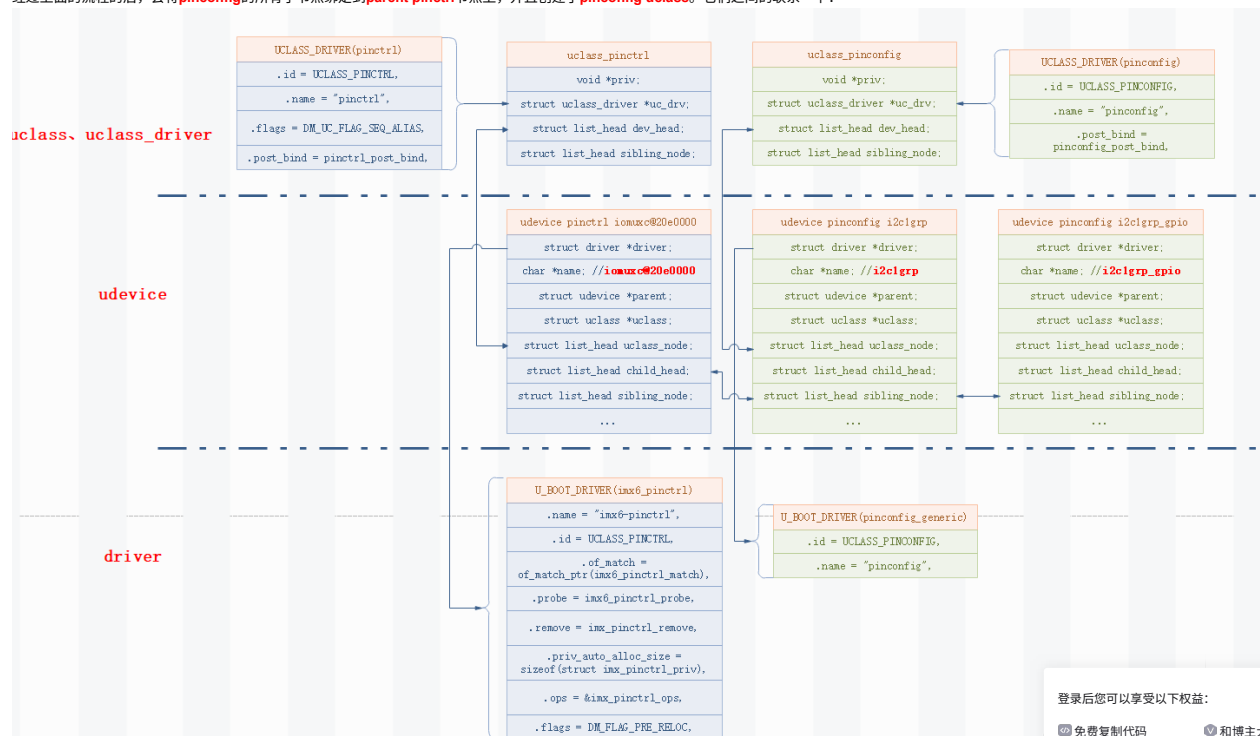


4. Pinconfig Uclass创建流程

pinctrl设备绑定后，调用pinctrl uclass driver的 `post_bind` 方法，来遍历pinctrl的子节点，并创建了的pinconfig uclass。具体的流程 下：

```
1 [UCLASS_DRIVER]uc->uc_drv->post_bind(dev); // 在一个新设备绑定到这个uclass后被调用
2 [pinctrl-uclass.c]pinctrl_post_bind(dev); // PINCTRL uclass正在post binding
3 [pinctrl-uclass.c]pinconfig_post_bind(dev); // dev = pinctrl(iomuxc@20e0000) 绑定它的子节点，以便在以后的dt解析期间外设设备可以轻松地地在父设备中搜索。
4 [list.c]device_bind_driver_to_node(dev, "pinconfig", "i2c1grp", node, NULL); // 将这个新设备绑定到一个给定设备树节点的驱动程序，只有在节点缺少"compatible"字符串时才需要这样做。
5 [device.c]device_bind_with_driver_data(parent, entry, name, id->data, node, &dev); // 创建一个设备并且绑定到driver。
6 [device.c]device_bind_common(parent, drv, name, NULL, driver_data, node, 0, devp); // 创建一个设备并且绑定到driver。
7 [uclass.c]uclass_get(drv->id, &uc); // 根据ID获取一个uclass， 果它不存在就创建它。
8 [uclass.c]uclass_find(); // 根据id找到对应的uclass。
9 [uclass.c]uclass_add(); // 在未找到的情况下，就会在列表中创建一个新的uclass。
10 [device.c]calloc(1, size); // 1: 创建一个新的udevice并初始化； 2: 给dev->platdata, dev->uclass_platdata, dev->parent_platdata分配空间
11 [device.c]list_add_tail(&dev->sibling_node, &parent->child_head); // 将新dev放到父节点的列表中
12 [uclass.c]uclass_bind_device(dev); // 将udevice与uclass进行绑定，设备连接到uclass的设备列表中。
13 [U_BOOT_DRIVER]drv->bind(dev); // device绑定成功后，就会调用drv->bind。
14 [U_BOOT_DRIVER]parent->driver->child_post_bind(dev); // 在一个新的child被绑定后，就会调用parent driver的child_post_bind(dev);
15 [UCLASS_DRIVER]uc->uc_drv->post_bind(dev); // 在一个新设备绑定到这个uclass后被调用
```

经过上面的流程的后，会将pinconfig的所有子节点绑定到parent pinctrl节点上，并且创建了pinconfig uclass。它们之间的联系 下：



在uboot的命令行，我们可以输入 `dm tree` 来打印它们之间的关系。通过DM Tree我们可以很直观的看到pinctrl和pinconfig uclass它们之间的关系，具体 下：



ZHONGCAI0901

关注

登录后您可以享受以下权益：

免费复制代码

和博主大V互动

下载海量资源

发动态/写文章/加入社区

立即登录

分享

=> dm tree				
Class	Index	Probed	Driver	Name

root	0	[+]	root_driver	root_driver
thermal	0	[]	imx_thermal	-- imx_thermal
simple_bus	0	[+]	generic_simple_bus	-- soc
simple_bus	1	[+]	generic_simple_bus	-- aips-bus@2000000
simple_bus	2	[]	generic_simple_bus	-- spba-bus@2000000
gpio	0	[+]	gpio_mxc	-- gpio@209c000
gpio	1	[+]	gpio_mxc	-- gpio@20a0000
gpio	2	[+]	gpio_mxc	-- gpio@20a4000
gpio	3	[+]	gpio_mxc	-- gpio@20a8000
gpio	4	[+]	gpio_mxc	-- gpio@20ac000
eth	0	[]	fecmxc	-- ethernet@20b4000
eth_phy_ge	0	[]	eth_phy_generic_drv	-- ethernet-phy@2
eth_phy_ge	1	[]	eth_phy_generic_drv	-- ethernet-phy@1
simple_bus	3	[]	generic_simple_bus	-- anatop@20c8000
simple_bus	4	[]	generic_simple_bus	-- snvs@20cc000
pinctrl	0	[+]	imx6-pinctrl	-- iomuxc@20e0000
pinconfig	0	[]	pinconfig	-- cs1grp
pinconfig	1	[+]	pinconfig	-- enet1grp
pinconfig	2	[+]	pinconfig	-- enet2grp
pinconfig	3	[]	pinconfig	-- flexcan1grp
pinconfig	4	[]	pinconfig	-- flexcan2grp
pinconfig	5	[]	pinconfig	-- i2c1grp
pinconfig	6	[]	pinconfig	-- i2c1grp_gpio
pinconfig	7	[]	pinconfig	-- i2c2grp
pinconfig	8	[+]	pinconfig	-- lcdifdatgrp
pinconfig	9	[+]	pinconfig	-- lcdifctrlgrp
pinconfig	10	[]	pinconfig	-- qspigrp
pinconfig	11	[]	pinconfig	-- sai2grp
pinconfig	12	[]	pinconfig	-- pwm1grp
pinconfig	13	[]	pinconfig	-- sim2grp
pinconfig	14	[]	pinconfig	-- spi4grp
pinconfig	15	[]	pinconfig	-- tscgrp
pinconfig	16	[]	pinconfig	-- uart1grp
pinconfig	17	[]	pinconfig	-- uart2grp
pinconfig	18	[]	pinconfig	-- usbotg1grp
pinconfig	19	[+]	pinconfig	-- usdhc1grp
pinconfig	20	[]	pinconfig	-- usdhc1grp100mhz
pinconfig	21	[]	pinconfig	-- usdhc1grp200mhz
pinconfig	22	[]	pinconfig	-- usdhc2grp
pinconfig	23	[+]	pinconfig	-- usdhc2grp_8bit
pinconfig	24	[]	pinconfig	-- usdhc2grp_8bit_100mhz
pinconfig	25	[]	pinconfig	-- usdhc2grp_8bit_200mhz
pinconfig	26	[]	pinconfig	-- wdoggrp
simple_bus	5	[+]	generic_simple_bus	-- aips-bus@2100000

5. I2C使用pinctrl功能

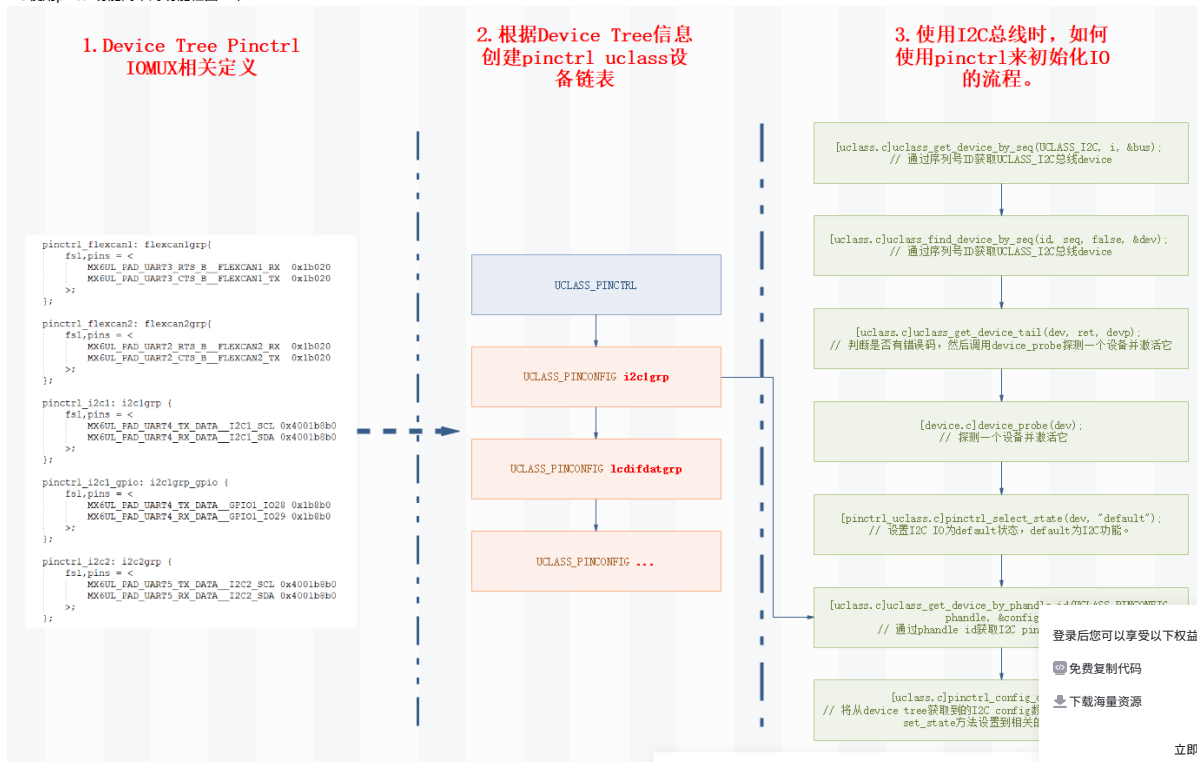
下面通过I2C使用pinctrl功能来举例，基本上分为三个步骤：

1. Device Tree定义了pinctrl I2C相关定义。
2. 根据Device Tree节点信息创建pinctrl uclass设备链表。
3. 当申请I2C总线时，会先probe设备激活设备，这样就会使用pinctrl里面pinconfig信息初始化IO。

涉及到的代码流程 下：

```
1 [uclass.c]uclass_get_device_by_seq(UCLASS_I2C, i, &bus); // 通过序列号ID获取UCLASS_I2C总线device
2 [uclass.c]uclass_find_device_by_seq(id, seq, false, &dev); // 通过序列号ID获取UCLASS_I2C总线device
3 [uclass.c]uclass_get_device_tail(dev, ret, devp); // 判断是否有错误码，然后调用device_probe探测一个设备并激活它
4 [device.c]device_probe(dev); // 探测一个设备并激活它
5 [pinctrl_uclass.c]pinctrl_select_state(dev, "default"); // 设置I2C IO为default状态，default为I2C功能。
6 [uclass.c]uclass_get_device_by_phandle_id(UCLASS_PINCONFIG, phandle, &config); // 通过phandle id获取I2C pinctrl的配置设备
7 [uclass.c]pinctrl_config_one(config); // 将从device tree获取到的I2C config数据，通过pinctrl driver的set_state方法设置到相关的寄存器上。
```

I2C使用pinctrl功能简单的功能框图 下：



Device Tree中I2C定义的信息 下：

```
1 | 6i2c1 {
2 |     clock-frequency = <100000>;
3 |     pinctrl-names = "default", "gpio";
4 |     pinctrl-0 = <&pinctrl_i2c1>;
5 |     pinctrl-1 = <&pinctrl_i2c1_gpio>;
6 |     scl-gpios = <&gpio1 28 GPIO_ACTIVE_HIGH>;
7 |     sda-gpios = <&gpio1 29 GPIO_ACTIVE_HIGH>;
8 |     status = "okay";
9 |
10 |     mag3110@e {
11 |         compatible = "fsl,mag3110";
12 |         reg = <0x0e>;
13 |     };
14 | };
15 |
16 | pinctrl_i2c1: i2c1grp {
17 |     fsl,pins = <
18 |         MX6UL_PAD_UART4_TX_DATA__I2C1_SCL 0x4001b8b0
19 |         MX6UL_PAD_UART4_RX_DATA__I2C1_SDA 0x4001b8b0
20 |     >;
21 | };
22 |
23 | pinctrl_i2c1_gpio: i2c1grp_gpio {
24 |     fsl,pins = <
25 |         MX6UL_PAD_UART4_TX_DATA__GPIO1_I028 0x1b8b0
26 |         MX6UL_PAD_UART4_RX_DATA__GPIO1_I029 0x1b8b0
27 |     >;
28 | };
```

6. Device Tree中phandle的使用

在Device Tree中解析时会用到phandle属性，我们需要了解一下它的使用。

```
1 | Property name: phandle
2 | Value type: <u32>
3 | Description:
4 | phandle属性为设备树中唯一的节点指定一个数字标识符。phandle属性值是被其它node使用，其它node通过phandle属性值与这个node进行关联。
```

Example:

参见以下devicetree引用：

```
1 | pic@10000000 {
2 |     phandle = <1>;
3 |     interrupt-controller;
4 | };
```

pic定义了一个phandle值为1，另一个设备节点可以用phandle值为1来引用pic节点：

```
1 | another-device-node {
2 |     interrupt-parent = <1>;
3 | };
```

注意：DTS中的大多数设备树不会包含显式的phandle属性。当DTS被编译成二进制DTB格式时，DTC工具会自动插入phandle属性。

8. 相关函数的使用

下面会具体分析所使用到的函数。

8.1 pinconfig_post_bind()函数分析

```
1 | UCLASS_DRIVER(pinctrl) = {
2 |     .id = UCLASS_PINCTRL,
3 |     .post_bind = pinctrl_post_bind,
4 |     .flags = DM_UC_FLAG_SEQ_ALIAS,
5 |     .name = "pinctrl",
6 | };
7 |
8 | /**
9 |  * pinctrl_post_bind() - PINCTRL uclass正在post binding
10 |  *
11 |  * 在full pinctrl的情况下，递归地将子节点pinconfig设备进行绑定。
12 |  */
13 | static int __maybe_unused pinctrl_post_bind(struct udevice *dev)
14 | {
15 |     const struct pinctrl_ops *ops = pinctrl_get_ops(dev); // 获取pin control操作指针
16 |
17 |     /**
18 |      * 果设置了set_state回调，我们假设这个pinctrl驱动程序就是完整的实现。
19 |      * 在这种情况下，应该绑定它的子节点，以便在以后的dt解析期间外设设备可以轻松地在父设备中搜索。
20 |      */
21 |     if (ops->set_state)
22 |         return pinconfig_post_bind(dev);
23 |
24 |     return 0;
25 | }
26 |
27 | /**
28 |  * pinconfig_post_bind() - PINCONFIG uclass正在post binding
29 |  * 递归地将其子设备pinconfig进行绑定。
30 |  *
31 |  pinctrl      0 [ + ] imx6-pinctrl      | |    |-- iomuxc@20e0000
32 |  pinconfig    0 [ ] pinconfig          | |    |-- csilgrp
33 |  pinconfig    1 [ + ] pinconfig          | |    |-- enet1grp
34 |  pinconfig    2 [ + ] pinconfig          | |    |-- enet2grp
35 |  pinconfig    3 [ ] pinconfig          | |    |-- flexcan1grp
36 |  pinconfig    4 [ ] pinconfig          | |    |-- flexcan2grp
37 |  pinconfig    5 [ ] pinconfig          | |    |-- i2c1grp
38 |
39 |  */
40 | static int pinconfig_post_bind(struct udevice *dev)
41 | {
42 |     bool pre_reloc_only = !(gd->flags & GD_FLG_RELOC);
43 |     const char *name;
44 |     ofnode node;
```

登录后您可以享受以下权益：

📄 免费复制代码

💬 和博主大V互动

📚 下载海量资源

✍️ 发动态/写文章/加入社区

立即登录



ZHONGCAI0901

关注

分享

```
45     int ret;
46
47
48     // 遍历该设备pinctrl的子节点pinconfig
49     dev_for_each_subnode(node, dev) {
50         if (pre_reloc_only &&
51             !ofnode_pre_reloc(node))
52             continue;
53
54         // 果该设备包含"compatible"属性, 那么这个节点不是一个pin config节点。但它是一个普通device, 所以这里跳过。
55         ofnode_get_property(node, "compatible", &ret);
56         if (ret >= 0)
57             continue;
58         // 果该节点的属性包含"gpio-controller", 则跳过。
59         if (ofnode_read_bool(node, "gpio-controller"))
60             continue;
61         // 果不是FDT_ERR_NOTFOUND, 则它不是一个pin config节点, 直接返回。
62         if (ret != -FDT_ERR_NOTFOUND)
63             return ret;
64
65         // 获取该节点的名称
66         name = ofnode_get_name(node);
67
68         ret = device_bind_driver_to_node(dev, "pinconfig", name, node, NULL);
69     }
70
71     return 0;
72 }
```

8.2 device_bind_driver_to_node()函数分析

```
1  /**
2   * device_bind_driver_to_node() - 将节点设备绑定到driver
3   *
4   * 这将一个新设备绑定到一个给定设备树节点的驱动程序, 只有在节点缺少"compatible"字符串时才需要这样做。
5   */
6  int device_bind_driver_to_node(struct udevice *parent, const char *drv_name,
7                                const char *dev_name, ofnode node,
8                                struct udevice **devp)
9  {
10     struct driver *drv;
11     int ret;
12
13     // 根据driver name查找对应的驱动程序。
14     drv = lists_driver_lookup_name(drv_name); // drv_name = "pinconfig"
15
16     /**
17      * device_bind_with_driver_data() - 创建一个设备并且绑定到driver。
18      * 设置一个新device连接到driver, 在这种情况下, 驱动通过提供driver_data的匹配表与设备匹配。
19      * 一旦绑定, 设备就存在, 但在调用 device_probe() 之前尚未激活。
20      */
21     ret = device_bind_with_driver_data(parent, drv, dev_name, 0 /* data */, node, devp); // parent = pinctrl, drv = pinconf_drv, dev_name= "i2c1grp", "cs1grp"...
22
23     return ret;
24 }
25
26
27 /**
28 * device_bind_with_driver_data() - 创建一个设备并且绑定到driver。
29 * 设置一个新device连接到driver, 在这种情况下, 驱动通过提供driver_data的匹配表与设备匹配。
30 * 一旦绑定, 设备就存在, 但在调用 device_probe() 之前尚未激活。
31 */
32 int device_bind_with_driver_data(struct udevice *parent,
33                                  const struct driver *drv, const char *name,
34                                  ulong driver_data, ofnode node,
35                                  struct udevice **devp)
36 {
37     return device_bind_common(parent, drv, name, NULL, driver_data, node,
38                               0, devp);
39 }
40
```

8.3 pinctrl_select_state_full()函数分析

```
1  /**
2   * pinctrl_select_state_full() - 真正实现了pinctrl_select_state的功能, 将设备设置为给定的状态
3   */
4  static int pinctrl_select_state_full(struct udevice *dev, const char *statename) // statename: "default"
5  {
6     char propname[32]; /* long enough */
7     const fdt32_t *list;
8     uint32_t phandle;
9     struct udevice *config;
10     int state, size, i, ret;
11
12     /**
13      * pinctrl-names = "default", "gpio";
14      * statename = "default";
15      * 通过dev_read_stringlist_search查找"default"的index是0, 所以这里state = 0。
16      */
17     state = dev_read_stringlist_search(dev, "pinctrl-names", statename); // state = 0
18
19
20     /**
21      * 因为state = 0, 所以这里propname = pinctrl-0。
22      * dev_read_prop函数功能是从设备节点读取一个属性, 它会获取到pinctrl-0的属性value。
23      * list是指向属性的指针, 果没有找到, 则为NULL。 list = &pinctrl_i2c1;
24      */
25     snprintf(propname, sizeof(propname), "pinctrl-%d", state);
26     list = dev_read_prop(dev, propname, &size);
27
28
29     size /= sizeof(*list);
30     for (i = 0; i < size; i++) {
31         phandle = fdt32_to_cpu(*list++);
32     }
33 }
```



ZHONGCAI0901

关注

登录后您可以享受以下权益:

📄 免费复制代码

💬 和博主大V互动

📁 下载海量资源

✍️ 发动态/写文章/加入社区

立即登录

分享

```

33     * uclass_get_device_by_phandle_id() - 通过phandle id获取一个uclass设备
34     */
35     ret = uclass_get_device_by_phandle_id(UCLASS_PINCONFIG, phandle,
36                                           &config);
37
38     ret = pinctrl_config_one(config);
39
40 }
41
42 return 0;
43 }
44
45 /**
46  * pinctrl_select_state() - 将设备设置为给定的状态
47  *
48  */
49 int pinctrl_select_state(struct udevice *dev, const char *statename)
50 {
51
52     if (!ofnode_valid(dev->node))
53         return 0;
54
55     if (pinctrl_select_state_full(dev, statename))
56         return pinctrl_select_state_simple(dev);
57
58     return 0;
59 }
60

```

8.4 uclass_get_device_by_phandle_id()函数分析

```

1  /**
2  * uclass_get_device_by_phandle_id() - 通过phandle id获取一个uclass设备
3  *
4  * 这将在uclass中的设备中搜索具有给定phandle id的设备。
5  *
6  * DTS中的大多数设备树不会包含显式的phandle属性。当DTS被编译成二进制DTB格式时，DTC工具会自动插入phandle属性。
7  *
8  * uclass_get_device_by_phandle_id(UCLASS_PINCONFIG, phandle, &config);
9  */
10 int uclass_get_device_by_phandle_id(enum uclass_id id, uint phandle_id,
11                                     struct udevice **devp)
12 {
13     struct udevice *dev;
14     struct uclass *uc;
15     int ret;
16
17     ret = uclass_get(id, &uc); // 获取UCLASS_PINCONFIG
18
19     /**
20      遍历UCLASS_PINCONFIG下所有的设备，并且匹配它的phandle id。
21      pinctrl      0 [ + ] imx6-pinctrl | | `-- iomuxc@20e0000
22      pinconfig    0 [ ] pinconfig      | | |-- csilgrp
23      pinconfig    1 [ + ] pinconfig      | | |-- enet1grp
24      pinconfig    2 [ + ] pinconfig      | | |-- enet2grp
25      pinconfig    3 [ ] pinconfig      | | |-- flexcan1grp
26      pinconfig    4 [ ] pinconfig      | | |-- flexcan2grp
27      pinconfig    5 [ ] pinconfig      | | |-- i2c1grp
28     */
29     uclass_foreach_dev(dev, uc) {
30         uint phandle;
31
32         phandle = dev_read_phandle(dev); // 获取当前设备在device tree node的phandle id
33         // 比较phandle id，是否和目标的一致。
34         if (phandle == phandle_id) {
35             *devp = dev;
36             // 判断是否有错误码，然后调用device_probe探测一个设备并激活它
37             return uclass_get_device_tail(dev, ret, devp);
38         }
39     }
40
41     return -ENODEV;
42 }

```

8.5 uclass_get_device_tail()函数分析

```

1  /**
2  * uclass_get_device_tail() - 判断是否有错误码，然后调用device_probe探测一个设备并激活它
3  */
4  int uclass_get_device_tail(struct udevice *dev, int ret, struct udevice **devp)
5  {
6      if (ret)
7          return ret;
8
9      // 探测一个设备并激活它
10     ret = device_probe(dev);
11     if (ret)
12         return ret;
13
14     *devp = dev;
15
16     return 0;
17 }

```

8.6 device_probe()函数分析

```

1  /**
2  * device_probe() - 探测一个设备并激活它
3  *
4  * 激活一个设备以便它可以随时使用。首先探查它的所有父节点。
5  */
6  int device_probe(struct udevice *dev)
7  {
8      const struct driver *drv;
9      int ret;
10     int seq;

```



ZHONGCAI0901

关注

登录后您可以享受以下权益：

免费复制代码

和博主大V互动

下载海量资源

发动态/写文章/加入社区

立即登录

分享

```

11
12 // 检测该device是否已经激活, 已激活就直接返回。
13 if (dev->flags & DM_FLAG_ACTIVATED)
14     return 0;
15
16 drv = dev->driver; // 获取该设备对应的driver
17
18 /**
19  * device_ofdata_to_platdata() - 设备读取平台数据
20  *
21  * 读取设备的平台数据(通常是从设备树中), 以便提供探测设备所需的信息。
22  */
23 ret = device_ofdata_to_platdata(dev);
24
25 // 果该设备存在parent, 那么先probe parent设备, 确保所有的parent dev都被probed。
26 if (dev->parent) {
27     ret = device_probe(dev->parent);
28
29     if (dev->flags & DM_FLAG_ACTIVATED)
30         return 0;
31 }
32
33 /**
34  * uclass_resolve_seq() - 解析device的序列号
35  *
36  * 在"dev->seq = -1"时, 然后"dev->req_seq = -1"代表自动分配一个序列号, "dev->req_seq > 0"代表分配
37  * 指定的序列号。 果请求的序列号正在使用中, 那么该设备将被分配另一个序列号。
38  *
39  * 注意, 该函数不会改变设备的seq值, dev->seq需要手动赋值修改。
40  */
41 seq = uclass_resolve_seq(dev);
42 dev->seq = seq;
43
44 // 标记该设备处于激活状态。
45 dev->flags |= DM_FLAG_ACTIVATED;
46
47 //处理除root device的pinctrl之外的所有设备, 对于pinctrl device不进行pinctrl的设置, 因为设备可能还没有被probed。
48 if (dev->parent && device_get_uclass_id(dev) != UCLASS_PINCTRL)
49     pinctrl_select_state(dev, "default");
50
51 /**
52  * uclass_pre_probe_device() - 处理一个即将被probed的设备
53  *
54  * 先执行uclass需要的任何预处理, 然后才可以探测它。这包括uclass的pre-probe()方法和父uclass的child_pre_probe()方法。
55  */
56 ret = uclass_pre_probe_device(dev);
57
58 if (dev->parent && dev->parent->driver->child_pre_probe) {
59     ret = dev->parent->driver->child_pre_probe(dev);
60 }
61
62 // 只处理具有有效ofnode的设备
63 if (dev_of_valid(dev) && !(dev->driver->flags & DM_FLAG_IGNORE_DEFAULT_CLKS)) {
64     // 处理{clocks/clock-parents/clock-rates}属性配置时钟
65     ret = clk_set_defaults(dev, 0);
66 }
67
68 // 执行该设备的driver的probe函数, 激活该设备。
69 if (drv->probe) {
70     ret = drv->probe(dev);
71 }
72
73 /**
74  * uclass_post_probe_device() - 处理一个刚刚被probed过的设备
75  *
76  * 对uclass探测设备后, 需要执行的操作。
77  */
78 ret = uclass_post_probe_device(dev);
79
80 if (dev->parent && device_get_uclass_id(dev) == UCLASS_PINCTRL)
81     pinctrl_select_state(dev, "default");
82
83 return 0;
84 }

```

8.7 pinctrl_config_one()函数分析

```

1 /**
2  * pinctrl_config_one() - 为单个节点应用pinctrl Settings
3  */
4 static int pinctrl_config_one(struct udevice *config)
5 {
6     struct udevice *pctldev;
7     const struct pinctrl_ops *ops;
8
9     pctldev = config;
10    for (;;) {
11        // 获取dev的父节点
12        pctldev = dev_get_parent(pctldev);
13        if (!pctldev) {
14            dev_err(config, "could not find pctldev\n");
15            return -EINVAL;
16        }
17        // 判断该dev的uclass是否是UCLASS_PINCTRL
18        if (pctldev->uclass->uc_drv->id == UCLASS_PINCTRL)
19            break;
20    }
21    // 获取设备driver的操作指针
22    ops = pinctrl_get_ops(pctldev);
23    return ops->set_state(pctldev, config); // 调用driver的set_state方法, 设置pinctrl到配置的状态。
24 }

```



ZHONGCAI0901

关注

登录后您可以享受以下权益：

免费复制代码

和博主大V互动

下载海量资源

发动态/写文章/加入社区

立即登录

分享

【Audio】基于STM32 I2S移植WM8978
Audio Codec驱动 13065

分类专栏

 HW Protocol

10篇

 Network Protocol

5篇

 LwIP

9篇

 MCU

10篇

 Other

2篇

 Qt

6篇

▼

最新评论

【FreeRTOS】基于STM32移植LWIP 2.1...
pooq3355: 解决了吗，我也是这个问题，能ping通，就是建立不了链接

【FreeRTOS】基于STM32移植LWIP 2.1...
嗨... 梦 J 叶、累*: 移植后能ping通,但是TCP/UDP连不上,怎么解决?

【Ethernet】以太网卡LAN8720A分析和...
qq_31785357: 你这ID是真的6

【Audio】I2S传输PCM音频数据分析总...
糖醋捞杖_8: 老哥，请问一下pcm打开文件的那个是？

【TOOLS】TortoiseSVN 何设置比较...
m0_54656857: 非常清晰明了👍

大家在看

TCP 协议设计入门：自定义消息格式与粘包解决方案 1

（通俗易懂）大模型部署避坑指南：资源、速度与实战要点解析

科普文：软件架构设计之应用安全【身份验证(Authentication)：短信验证、图片验证、TOTP一次性密码、2FA双重身份验证等小结】 935

未来五年最抢手的AI岗位：揭秘年薪百万的产品经理新物种！

【天文学】海王星

最新文章

【Docker】ubuntu20.04 X86机器搭建NVIDIA ARM64 TX2的Docker镜像

【I2C】Linux使用GPIO模拟I2C

【I2C】Linux I2C子系统分析

2023年 7篇 2022年 7篇

2021年 37篇 2020年 23篇

2019年 9篇

目录

相关文章

1. 前言

2. Pinctrl子系统的功能简介

3. Pinctrl Uclass创建流程

4. Pinconfig Uclass创建流程

5. I2C使用pinctrl功能

6. Device Tree中phandle的使用

8. 相关函数的使用

8.1 pinconfig_post_bind()函数分析

8.2 device_bind_driver_to_node()函...

8.3 pinctrl_select_state_full()函数分析

8.4 uclass_get_device_by_phandle_...

8.5 uclass_get_device_tail()函数分析

登录后您可以享受以下权益：

 免费复制代码

 和博主大V互动

 下载海量资源

 发动态/写文章/加入社区

立即登录